

第二阶段工作小结

前言

本阶段是在第一阶段项目骨架已经搭建完成的基础上继续推进的。第一阶段解决的是“项目能不能稳定跑起来”的问题，包括前后端工程初始化、Prisma 与 SQLite 接入、健康检查接口、前端状态页以及基础运行说明。本阶段则进一步往业务主链路推进，重点解决“系统是否已经具备可演示、可联调、可继续扩展的核心能力”。

当前项目仍然处在课程项目的早期开发阶段，因此本阶段没有直接接入真实大语言模型，也没有接入真实天气服务，更没有一次性把所有旅游业务功能全部做完。我们的策略是先把用户、偏好、历史记录和单城市规划这几条关键线索串起来，用 Mock 规划逻辑打通完整链路。这样做的好处是非常明显的：后续真正接入大模型时，不需要重新设计用户体系、历史保存、接口调用方式和前端展示流程，只需要替换规划生成服务内部的实现。

从结果来看，项目已经从“能运行的基础骨架”推进到“能演示核心主链路”的状态。用户现在可以注册、登录、创建偏好卡片、生成一份单城市 Mock 旅行方案，并在历史记录中看到系统自动保存的规划结果。这说明项目已经不再只是孤立的前端页面或后端接口，而是具备了前端、后端、数据库三者协同工作的完整雏形。

1. 本阶段工作内容，以及之前已取得的开发成果回顾

1.1 第一阶段成果回顾

第一阶段的主要目标是搭建项目地基，让项目具备最小可运行能力。这个阶段虽然没有实现旅游业务功能，但它为后续开发提供了非常重要的基础。

项目采用前后端分离结构，根目录下保留两个独立工程：

- frontend：前端工程，使用 React + TypeScript + Vite + Tailwind CSS。
- backend：后端工程，使用 Node.js + Express + TypeScript + Prisma + SQLite。

前端部分完成了 Vite 项目初始化、React 启动入口、Tailwind CSS 样式接入，以及最基础的状态展示页面。这个页面一开始只承担一个职责：请求后端 GET /api/health 接口，并把后端和数据库状态展示出来。它不是正式业务首页，而是开发阶段的联调入口。

后端部分完成了 Express 应用初始化，拆分了 `app` 和 `server`，并建立了基础路由结构。`GET /api/health` 接口不仅返回静态状态，还会通过 Prisma 对 SQLite 做一次最小数据库探测，从而确认数据库链路真实可用。这个设计比单纯返回 `ok` 更可靠，因为它能同时验证后端服务、路由、中间件和数据库连接。

数据库方面，第一阶段完成了 Prisma 与 SQLite 的基础接入，并明确了后续必须走标准 migration 流程。也就是说，后续数据库结构变更不直接手动改 SQLite 文件，而是在 `schema.prisma` 中定义模型，再通过 Prisma migration 生成数据库结构变化记录。这一点对课程项目很重要，因为它能让数据库演进过程有迹可循，避免后期字段越来越多时无法判断结构来源。

整体来看，第一阶段的价值是把项目从“空目录”推进到“前端、后端、数据库可以互相连通”的状态。这个基础虽然看起来朴素，但它直接决定了第二阶段能不能顺利开发业务能力。

1.2 本阶段新增核心数据模型

第二阶段首先补齐了后续业务需要的核心数据库模型，主要包括三张表：

- `User`：用户账号表，保存用户名、密码摘要、创建时间和更新时间。
- `PreferenceCard`：偏好卡片表，保存用户的旅行偏好，例如旅行风格、交通方式、景点偏好、出发城市、同行类型、旅行天数、天气参考方式等。
- `TravelRecord`：历史记录表，保存用户生成过的旅行方案，包括记录类型、输入摘要、结果标题、完整方案内容、天气信息和可选关联的偏好卡片。

这三张表的关系也已经建立起来：

- 一个用户可以拥有多张偏好卡片。
- 一个用户可以拥有多条历史记录。
- 一条历史记录可以可选关联一张偏好卡片。
- 删除用户时，级联删除该用户的偏好卡片和历史记录。
- 删除偏好卡片时，不删除历史记录，只把历史记录上的 `cardId` 置空。

这里的设计重点是保护历史记录。偏好卡片是用户当前偏好的表达，它可能会被修改或删除；而历史记录代表过去已经生成过的方案，应该尽量保留。即使某张偏好卡片被删除，用户也仍然可以回看之前的规划结果。这种处理方式更符合真实产品逻辑，也避免了用户误删偏好卡片时连带丢失历史方案。

1.3 完成注册与登录接口

在有了 `User` 模型之后，本阶段实现了后端最基础的认证能力，主要包括：

- `POST /api/auth/register`
- `POST /api/auth/login`

- GET /api/auth/me

注册接口负责创建用户，接收用户名、密码和确认密码。密码不会以明文形式保存，而是通过哈希摘要处理后写入 passwordHash 字段。这个字段命名也刻意避免使用 password，因为 password 容易让人误以为数据库中保存的是明文密码。即使是课程项目，也应该从一开始养成不保存明文密码的习惯。

登录接口负责校验用户名和密码。登录成功后返回用户基础信息和 JWT token。这里返回的用户信息只包含安全字段，例如 id、username、createdAt，不会返回 passwordHash。

GET /api/auth/me 是一个受保护接口，用来验证当前 token 是否有效，并返回当前登录用户信息。这个接口非常适合前端刷新页面后恢复登录态，因为前端只要从 localStorage 取出 token，再请求 /api/auth/me，就能知道当前用户是否仍然有效。

1.4 建立 JWT 登录态与认证中间件

本阶段在后端引入了 JWT 登录态，但控制了范围，没有引入 session、cookie、刷新 token、角色权限等复杂设计。

JWT 的作用是让后端能够识别“当前请求是谁发来的”。登录成功后，后端生成 token 返回给前端。前端后续请求受保护接口时，在请求头中携带：

Authorization: Bearer <token>

后端认证中间件会读取并校验这个 token。如果 token 有效，就把当前用户信息挂到请求对象上，后续路由和 service 就可以通过当前用户 ID 操作数据。如果 token 缺失、格式错误、无效或过期，则统一返回 401。

这个设计为后续所有“当前用户自己的数据”提供了基础。偏好卡片、历史记录、规划接口都依赖这个认证中间件来判断用户身份。

1.5 完成偏好卡片后端 CRUD

偏好卡片是后续旅游规划的重要输入。本阶段实现了登录用户自己的偏好卡片增删改查能力，接口统一挂载在 /api/cards 下：

- GET /api/cards：查询当前用户的偏好卡片列表。
- POST /api/cards：创建偏好卡片。
- PUT /api/cards/:id：更新指定偏好卡片。
- DELETE /api/cards/:id：删除指定偏好卡片。

这些接口都有一个共同原则：只能操作当前登录用户自己的数据。也就是说，用户 A 即使知道用户 B 的卡片 ID，也不能查看、修改或删除用户 B 的偏好卡片。后端不会只依赖前端隐藏按钮来保证安全，而是在数据库查询条件中同时使用 `id` 和 `userId`，从根源上避免越权访问。

创建和更新偏好卡片时，也做了基础参数校验。例如必填字符串不能为空，`travelDays` 必须是正整数，`startDate` 如果传入则必须能解析为合法日期。同时，本阶段限制每个用户最多创建 10 张偏好卡片，避免早期开发阶段产生过多无意义数据。

1.6 完成前端偏好卡片管理面板

后端偏好卡片接口完成后，前端也新增了对应的管理面板。登录成功后，用户可以在当前单页中看到偏好卡片区域，并完成以下操作：

- 查看自己的偏好卡片列表。
- 创建新的偏好卡片。
- 编辑已有偏好卡片。
- 取消编辑。
- 删除偏好卡片。
- 刷新列表。

这一阶段没有引入路由库、状态管理库或组件库，而是继续保持单页结构。这样做是有意控制复杂度，因为当前阶段的主要目标是验证业务链路，而不是立刻把页面拆成正式产品结构。

虽然页面仍然是开发验证性质，但偏好卡片已经是真实写入数据库的，不是纯前端假数据。这意味着它已经被后续规划接口复用。

1.7 完成历史记录后端接口

历史记录是旅游规划系统非常关键的功能。用户生成过的方案如果不能保存，那么系统体验会很弱，也不利于后续做方案优化、继续编辑、分享图生成等能力。

本阶段实现了 `/api/history` 接口组：

- `GET /api/history`：查询当前用户历史记录列表。
- `GET /api/history/:id`：查看某条历史记录详情。
- `POST /api/history`：创建历史记录。
- `PUT /api/history/:id/title`：修改历史记录标题。
- `DELETE /api/history/:id`：删除历史记录。

这些接口同样都受 JWT 保护，并且只操作当前用户自己的历史记录。用户 A 不能访问、修改或删除用户 B 的历史记录。

其中 `POST /api/history` 当前主要用于联调和后续复用。真正的用户场景中，历史记录不一定是用户手动创建的，而更可能是在“旅游规划生成成功”之后由系统自动写入。因此本阶段先把历史记录 service 做出来，为后续规划接口复用打基础。

1.8 完成前端历史记录面板

前端新增了历史记录面板，登录后显示在页面下方。用户可以：

- 查看历史记录列表。
- 创建测试历史记录。
- 查看历史记录详情。
- 修改历史记录标题。
- 删除历史记录。

历史详情中的完整方案内容采用内联展示方式，没有做弹窗或独立详情页。这同样符合当前阶段的目标：先验证链路，暂时不做过度 UI 设计。

这个面板的意义不只是展示历史记录本身，更重要的是它为后续规划功能提供了验证入口。只要规划接口生成结果后能自动保存历史，用户就能立刻在历史记录区域看到生成结果。

1.9 完成单城市 Mock 规划闭环

本阶段最后打通了单城市规划的最小闭环，新增了受 JWT 保护的接口：

`POST /api/plan/city`

这个接口支持以下输入：

- `targetCity`：目标城市。
- `departureCity`：出发城市。
- `travelDays`：旅行天数。
- `cardId`：可选偏好卡片 ID。
- `temporaryPreference`：临时偏好。
- `weatherMode`：天气参考方式。

如果用户传入 `cardId`，后端会先校验这张偏好卡片是否属于当前用户。如果不属于当前用户或不存在，则返回 `404`。如果校验通过，后端会读取偏好卡片中的旅行风格、交通方式、景点偏好、同行类型、旅行天数等字段，用这些信息参与 Mock 方案生成。

如果用户不传 `cardId`，后端则使用请求中的临时偏好和基础字段生成方案。

当前生成的方案不是由真实大语言模型生成，而是使用固定模板拼接出结构化文本，内容包括：

- 方案标题。
- 方案概览。
- 每日安排。
- 交通建议。
- 天气提示。
- 注意事项。

生成成功后，后端会自动创建一条 `TravelRecord`，其中 `recordType` 固定为 `single-city-plan`。接口最终返回 `plan + record`，也就是既返回本次生成的方案，也返回已经保存到数据库的历史记录。

这一步是本阶段最重要的闭环。它说明系统已经具备了“提交规划请求 -> 后端生成结果 -> 保存历史记录 -> 前端展示结果 -> 历史面板刷新”的完整路径。虽然规划内容还是 Mock，但主流程已经基本成型。

1.10 当前可以实际体验的功能链路

目前用户可以按照以下顺序实际进入页面体验：

1. 启动后端服务。
2. 启动前端服务。
3. 打开前端页面。
4. 查看健康检查结果，确认后端和数据库正常。
5. 注册一个新账号。
6. 使用账号登录。
7. 创建一张偏好卡片。
8. 使用临时偏好或偏好卡片生成单城市 Mock 方案。
9. 在页面中查看生成结果。
10. 到历史记录面板中查看自动保存的规划记录。

这条链路已经覆盖了用户、偏好、规划、历史四个核心业务概念。对于一个智能旅游规划系统来说，这已经是非常有价值的阶段成果。

2. 遇到的问题以及解决思路

2.1 问题一：如何控制开发范围，避免功能一次性铺太大

现象

智能旅游规划系统天然包含很多功能，例如登录注册、偏好卡片、单城市攻略、多城市自驾路线、历史记录、方案优化、天气信息、分享图、大语言模型调用等。如果一开始就同时推进所有功能，很容易导致前后端接口不稳定、数据库模型频繁改动、页面结构混乱。

原因分析

课程项目最容易出现的问题不是“功能太少”，而是“每个功能都开了头，但没有一条链路真正打通”。如果在数据模型还没稳定时就开始做复杂页面，在认证还没完成时就写用户数据接口，在历史记录还没设计好时就接大模型，后续返工成本会很高。

解决思路

本阶段采用了“最小闭环优先”的策略。每一步只做当前阶段真正需要的能力：

- 先做用户模型，再做注册登录。
- 先做 JWT 登录态，再做用户私有数据。
- 先做偏好卡片和历史记录，再做规划接口。
- 先做 Mock 规划，再考虑真实大语言模型。

这种顺序保证了每一个新增能力都能被前面的基础支撑，也能为后面的功能复用。

当前结果

项目没有陷入“大而全但都不完整”的状态，而是形成了一条可实际演示的主链路。当前虽然还没有真实智能生成能力，但系统结构已经能承载后续大模型接入。

2.2 问题二：Prisma 与 SQLite 如何保持标准 migration 流程

现象

在课程项目中，SQLite 很容易被当成一个可以随手改的本地文件。开发者可能直接删除数据库、手动改表，或者跳过 migration。这种做法短期看很快，但后续会导致数据库结构不可追踪。

原因分析

ORM 的价值不只是让代码少写 SQL，更重要的是让数据库结构和代码模型保持一致。Prisma 的 `schema.prisma` 是数据库模型的源头，migration 是结构变化的记录。如果直接手动改 SQLite 文件，团队成员或后续自己都很难知道数据库到底经历了哪些变化。

解决思路

本阶段在设计 `User`、`PreferenceCard`、`TravelRecord` 时，采用了标准 Prisma migration 流程。也就是先在 `schema.prisma` 中定义模型和关系，再通过 migration 应用到 SQLite。数据库文件本身仍然是本地开发产物，不作为主要结构来源。

同时，后续阶段如果需要新增字段或新增模型，也应该继续走：

```
npm run prisma:migrate:dev -- --name <change-name>
```

部署或正式环境则使用：

```
npm run prisma:migrate:deploy
```

当前结果

当前数据库已经有清晰的核心模型和关系，后续功能可以基于这三张表继续扩展。更重要的是，项目已经建立了正确的数据库演进习惯，不会因为短期省事破坏长期可维护性。

2.3 问题三：JWT 登录态应该做到什么程度

现象

实现登录功能时，有很多可选方案。例如只返回用户信息、使用 session、使用 cookie、使用 JWT、增加刷新 token、增加角色权限、增加前端全局状态管理等。如果一开始全部实现，会明显超出当前阶段范围。

原因分析

认证系统本身可以非常复杂，但当前课程项目这个阶段只需要解决一个核心问题：后端如何知道当前请求来自哪个用户。只要能可靠识别当前用户，就可以继续开发偏好卡片、历史记录和规划接口。

解决思路

本阶段选择使用最小 JWT 登录态。登录成功后返回 token，前端保存在 localStorage 中。后续请求受保护接口时，在 Authorization 请求头中携带 token。后端通过认证中间件校验 token，并把用户信息挂到请求对象上。

暂时没有实现刷新 token、角色权限、管理员体系、cookie session 等能力。这不是遗漏，而是刻意控制范围。等到后续项目进入更正式的用户体验阶段，再考虑更严格的安全策略。

当前结果

当前认证能力已经足够支撑用户私有数据访问。偏好卡片、历史记录和规划接口都可以基于当前登录用户工作。前端刷新页面后，也能通过 /api/auth/me 恢复当前用户状态。

2.4 问题四：如何确保用户只能操作自己的数据

现象

一旦系统有了多用户，就会出现数据隔离问题。例如用户 A 创建了一张偏好卡片，用户 B 如果知道这张卡片的 ID，是否能修改或删除它？用户 B 是否能用用户 A 的 cardId 生成规划？这些问题如果处理不好，会造成典型的越权漏洞。

原因分析

前端隐藏按钮或不展示别人的数据，并不能真正保证安全。因为接口可以被直接请求，用户可以绕过页面发送任意 ID。真正的数据隔离必须在后端完成。

解决思路

本阶段所有用户私有数据接口都遵循同一个原则：数据库查询条件必须同时带上资源 ID 和当前用户 ID。例如查询偏好卡片时，不只按 id 查询，还要按 userId 查询。历史记录也是一样。

单城市规划接口在使用 cardId 时，也会校验这张偏好卡片是否属于当前用户。其他用户传入这张卡片 ID 时，后端返回 404，而不是泄露这张卡片存在但无权限。

当前结果

当前系统已经具备基础的数据隔离能力。用户只能看到、修改、删除、使用自己的偏好卡片和历史记录。这为后续继续开发更多用户数据接口打下了安全边界。

2.5 问题五：Mock 规划如何为后续真实大模型接入保留扩展口

现象

当前阶段还没有接入真实大语言模型，但系统又需要尽早验证“旅游规划”这条主链路。如果完全不做规划接口，历史记录和偏好卡片无法形成闭环；如果过早接入真实 LLM，又会引入 API Key、网络调用、费用、错误处理、提示词设计等复杂问题。

原因分析

大语言模型接入不是单纯调用一个接口。真实场景下还要考虑提示词结构、输入清洗、超时处理、失败重试、返回内容解析、模型费用、隐私信息、日志记录等。如果在基础业务链路还没稳定时就直接接入，问题会混在一起，很难判断是业务设计问题，还是模型调用问题。

解决思路

本阶段先实现 `POST /api/plan/city` 的 Mock 版本。接口形状、输入字段、权限校验、历史记录保存流程都按照未来真实规划接口来设计，但生成内容暂时由固定模板完成。

这样做相当于先把插座、线路和开关装好，灯泡暂时用一个测试灯。等后续接入真实大模型时，只需要重点替换后端 `plan service` 内部的生成逻辑，而不用重做前端表单、接口权限、历史记录保存和结果展示。

当前结果

单城市规划已经形成完整闭环。Mock 内容虽然不是智能生成，但接口和数据流已经接近真实业务流程。这个设计让项目可以稳步过渡到真实 LLM 阶段，而不是推倒重来。

3. 下一阶段计划

3.1 优先推进多城市自驾路线 Mock 闭环

下一阶段建议优先实现多城市自驾路线的 Mock 规划能力。原因是系统设计文档中已经明确包含“多城市自驾路线”这一核心功能，而当前单城市 Mock 规划已经证明了规划接口和历史记录保存流程可行。

多城市自驾路线可以继续沿用当前模式：

- 新增受 JWT 保护的规划接口。
- 接收出发城市、途经城市、目的城市、旅行天数、偏好卡片 ID 或临时偏好。
- 后端先校验用户身份和偏好卡片归属。
- 使用 Mock 模板生成自驾路线建议。

- 自动保存到 TravelRecord 。
- 前端增加一个轻量面板用于输入和展示结果。

这一阶段仍然不建议直接接入真实大模型。因为多城市路线涉及更多字段和更复杂的输入结构，先用 Mock 版本验证数据流会更稳。

3.2 逐步整理前端页面结构

当前前端仍然是一个单页验证页面，所有能力都集中在首页中，包括健康检查、注册登录、偏好卡片、单城市规划、历史记录等。这个结构适合早期联调，但随着功能变多，页面会越来越长，代码也会越来越难维护。

下一阶段或再下一阶段可以开始考虑引入正式页面结构，例如：

- 登录注册页。
- 首页或仪表盘。
- 偏好卡片管理页。
- 单城市规划页。
- 多城市路线页。
- 历史记录页。

这时可以考虑引入 react-router，但不建议过早引入复杂状态管理库。当前项目的数据量和交互复杂度还不高，很多状态仍然可以通过组件内部状态和 API 请求解决。

3.3 规划真实大语言模型接入

当单城市和多城市 Mock 链路都稳定后，就可以开始规划真实大语言模型接入。这个阶段需要重点处理：

- API Key 存放方式。
- 后端调用模型接口，而不是前端直接调用。
- 提示词模板设计。
- 用户输入到提示词的映射。
- 模型返回内容的格式约束。
- 超时和失败处理。
- 模型调用失败时的降级提示。
- 调用结果保存到历史记录。

建议一开始只把真实 LLM 接入单城市规划，不要同时接所有规划类型。先把一个功能做扎实，再把经验复制到多城市路线和方案优化中。

3.4 接入天气信息能力

天气信息可以作为规划生成的辅助输入，而不是主流程的强依赖。也就是说，天气服务失败时，不应该导致整个规划生成失败。比较稳妥的策略是：

- 用户选择是否参考天气。
- 后端根据目标城市和日期尝试获取天气信息。
- 获取成功则注入规划上下文。
- 获取失败则使用提示文案告知用户天气暂不可用。
- 无论天气是否成功，旅游方案主流程尽量继续执行。

这种设计更符合真实系统的容错思路。外部服务不稳定是常态，核心业务流程应该尽量避免被非核心外部依赖完全阻断。

3.5 增加方案优化和分享图生成

在规划生成和历史保存稳定之后，可以继续做方案优化和分享图生成。

方案优化可以基于已有历史记录实现。用户选择某条历史记录，输入优化要求，例如“减少步行”“增加美食”“更适合亲子”“预算更低”，后端基于原始方案 and 用户要求生成新的优化版本，并再次保存为历史记录。

分享图生成可以先从前端截图或简单模板开始，不一定一开始就做复杂图片服务。课程项目阶段更重要的是展示“方案可以被整理和分享”的能力，而不是追求非常复杂的海报系统。

3.6 补充接口测试和开发文档

随着接口数量增加，后续需要逐步补充更系统的测试和文档。建议至少整理：

- 后端接口清单。
- 请求和响应示例。
- 常见错误码说明。
- 本地启动步骤。
- 数据库 migration 使用说明。
- 阶段性测试用例。

当前项目已经具备一定复杂度，如果不及时整理，后面很容易出现“接口能用，但不知道怎么用”的问题。文档不是最后答辩前才补的材料，而是开发过程中降低沟通成本的工具。

结语

本阶段最大的价值，是把项目从第一阶段的“能跑起来”推进到了“能演示主链路”。现在系统已经具备用户注册登录、JWT 登录态、偏好卡片管理、历史记录管理和单城市 Mock 规划生成能力。用户可以从登录开始，一路完成偏好录入、方案生成和历史回看，这说明前端、后端和数据库已经形成了真实协作。

当然，当前系统还不是最终形态。它还没有接入真实大语言模型，没有接入真实天气服务，也没有完成正式的多页面产品结构。但从工程角度看，现在的基础是健康的：数据模型已经建立，接口边界比较清晰，用户数据隔离已经考虑，历史记录保存流程已经打通，Mock 规划也为后续真实智能生成预留了替换空间。

下一阶段继续推进时，建议仍然坚持“最小可运行闭环”的思路。不要急着一次性把所有功能堆满，而是围绕一条条真实用户链路逐步增强。这样项目既能稳定前进，也能在课程展示时清楚说明每个阶段做了什么、为什么这样做，以及后续如何扩展。